

Part 1 – Concept Understanding

What's Expected

1. What are Web Vitals vs Core Web Vitals

Web Vitals:

Web Vitals are general and broader performance metrics through which we can measure the performance of our websites. These metrics do not impact the SEO ranking score.

Example: TTFB,FCP,INP,SI

Core Web Vitals:

Core web vitals are subset of web vitals that google considers absolutely *essential* to every web user's experience These metrics have a direct impact on the SEO ranking score

It represents **three core pillars** of the user **experience**:

Loading Performance
Interactivity/Responsiveness
Visual Stability

Example: LCP,INP,CLS

2. FCP (First Contentful Paint):

what it measures:

It measures the exact moment when a browser renders the first piece of the dom content such as text,images,svg and canvas

“good” threshold:

Good: ≤ 1.8 seconds

Needs Improvement: Between 1.8 seconds and 3.0 seconds

Poor: > 3.0 seconds

Common Causes of Poor FCP:

High Server Response Time (TTFB)
Render-Blocking JavaScript and CSS
Unoptimized Web Fonts
Inefficient DOM Size

3. LCP (Largest Contentful Paint):

what it measures:

LCP measures the time from when the user initiates loading the page to when the largest text block or image element within the viewport is fully rendered on the screen

“good” threshold:

Good: ≤ 2.5 seconds

Needs Improvement: Between 2.5 seconds and 4.0 seconds

Poor: > 4.0 seconds

Common Causes of Poor LCP:

Slow Server Response Times
Render-Blocking JavaScript and CSS
Resource Load Delay
Slow Resource Load Duration

4. TBT (Total Blocking Time):

what it measures:

TBT measures how unresponsive a page is while it is loading and becomes fully available for interaction with the user. It measures the amount of the time between FCP and the TTI where the main thread is blocked.

“good” threshold:

Good: ≤ 200 milliseconds

Needs Improvement: Between 200 milliseconds and 600 milliseconds

Poor: > 600 milliseconds

Common Causes of Poor TBT:

Heavy JavaScript Bundles
Long-Running Evaluation Tasks
Inefficient Third-Party Scripts
Large DOM Redraws

5. CLS (Cumulative Layout Shift):

what it measures:

CLS measures the total sum of all the unexpected layout shift scores for every visual element that moves while the page is open.

It is calculated as the product of the impact fraction and the distance fraction.

“good” threshold:

Good: ≤ 0.1

Needs Improvement: Between 0.1 and 0.25

Poor: > 0.25

Common Causes of Poor CLS:

Images without Dimensions
Ads, Embeds, and Iframes without Reserved Space
Dynamically Injected Content
Web Fonts (FOUT/FOIT)

6. SI (Speed Index):

what it measures:

Measures the speed at which a website populates its contents to make the viewport look like it is 100% completed.

Example:

Website A which loads 90% of its page in 4 seconds will have a higher speed index than Website B which loads 50% of its page in the same 4 seconds.

“good” threshold:

Good: ≤ 3.4 seconds

Needs Improvement: Between 3.4 seconds and 5.8 seconds
Poor: > 5.8 seconds

Common Causes of Poor SI:

Main Thread Contention
Large, Unoptimized Critical CSS
Slow Resource Responses

7. Lab data (Lighthouse) vs Field data (CrUX/RUM):

Lab data (Lighthouse):

These are the data collected under the controlled environments such as auditing a website using chrome's lighthouse. It creates a standardized environment with a standard network and tests how our website performs on that. It is useful for debugging and local development but even highly scored lab data can perform poorly when tested on a real user's device. It captures only the initial page load cycle.

Example: FCP,TTFB,INP,LCP,SI

Field data (CrUX/RUM):

These are the data collected from the actual real world users who have poor network connection and poor device configurations than the standard ones we tested with the lighthouse. It captures the entire lifecycle of the page including late layout shifts and interaction to next points. It can be analyzed using Google's chrome user experience report or real user monitoring custom scripts embedded inside our code.

Example: LCP,CLS,INP.

1. Metrics Table

Site	Device	FCP	LCP	TBT	CLS	SI	PerformanceScore	Accessib ility	Best Practices	SEO
https://www.fleetsstudio.com/	Mobile	4.7 s	6.2 s	20 ms	0 s	5.9 s	64	83	96	82
https://www.fleetsstudio.com/	Desktop	1.2 s	1.9 s	0 ms	0 s	1.8 s	86	83	100	82
https://www.wikipedia.org/	Mobile	1.3 s	1.3 s	0 ms	0	1.3 s	100	95	100	100
https://www.wikipedia.org/	Desktop	0.4 s	0.4 s	0 ms	0	0.4 s	100	100	100	100

2. Top Issues

Fleet Studio Desktop:

Reduce unused JavaScript
Image elements do not have explicit width and height
Reduce unused CSS

Fleet Studio Mobile:

Reduce unused CSS
Render-blocking requests
Reduce unused JavaScript

Wikipedia Desktop and Mobile:

Not any big issues and there are only minor improvements that can be made

Suboptimal Asset Cache Lifetimes
Long Main-Thread Interactivity Tasks

3. Root-Cause-Notes:

First Contentful Paint (FCP):

fleetstudio.com (Mobile): Poor (4.7s). Slow server response and heavy code files block the page from showing anything quickly.

wikipedia.org: Instant. Loads immediately because the page is incredibly lightweight.

Largest Contentful Paint (LCP):

fleetstudio.com (Mobile): Slow (6.2s). Large image files and unoptimized code delay the main part of the page from loading.

Total Blocking Time (TBT):

Both Sites: good (0–20ms). The screen never freezes because neither site uses heavy background scripts or annoying ads during startup.

Cumulative Layout Shift (CLS):

Both Sites: Perfect (0). Content doesn't unexpectedly jump around while loading because the layouts are simple and static.

Speed Index (SI):

fleetstudio.com (Mobile): Slow (5.9s). The top half of the screen stays blank for too long because the browser is stuck waiting to process the page styling.

4. Recommendations

1. Prioritize Critical Code (Fixes FCP)

- **What to do:** Take only the minimum styling needed for the top of the page and load it first. Delay all non essential scripts so they don't block the page from displaying immediately.

2. Clean Up Unused Code (Improves SPI)

- **What to do:** Delete old, unused CSS and JavaScript from the system. Minifying and cleaning up this code will make the website lightweight and load faster, increasing SPI.

3. Optimize Images (Fixes LCP)

- **What to do:** Always compress images so that the image can be loaded fast and the LCP can be recognized faster. Always define width and height dimensions in the code so the browser knows exactly how much space to save.

5. The most Impactful First Recommendation

This would be to optimize the FCP first which automatically improves the LCP eventually helping on the SPI and making the overall web vitals score higher improving the web performance.

